



Reporte de proyecto: Simulador de exámenes

- APLICACIÓN EN CPP

- UNIVERSIDAD AUTÓNOMA DE AGUASCALIENTES
- CARRERA: ISC
- SEMESTRE: 2
- MATERIA: ESTRUCTURA DE DATOS I
- PROFESOR: EDUARDO SERNA-PÉREZ
- FECHA DE ENTREGA: 24/06/26
- GRUPO: 2° A

```
|=====SIMULADOR DE EXAMENES=====|
|-----MENU-----|
> GENERAR UN EXAMEN
MODIFICAR UN EXAMEN
APLICAR UN EXAMEN
SALIR
```

- Isaac Tejeda
- Edgar Jehonadab Armas De Paz

REPOSITORIO: <https://github.com/isaactejed27-maker/SIMULADOR-DE-EXAMENES.git>

RESUMEN DESCRIPTIVO

En este apartado redactamos las fortalezas y debilidades al desarrollar este proyecto, para esto lo explicaremos en los 2 apartados antes mencionados:

FORTALEZAS

Comenzando por decir que todo funciona correctamente.

El código está ordenado.

Se empleó la herramienta 'github', para mantener un control de versiones y poder retornar en caso de un fallo.

Cumple con los requisitos pedidos en la descripción del proyecto, que son:

- memoria dinámica.
- Estructura de datos.
- Manejo de archivos.

Se utiliza una lista doblemente enlazada para desplazarse entre las preguntas dentro del examen.

Se hace el uso del formato pedido para los archivos de texto para los reactivos.

Se emplea el uso de la tabla de código ASCII, esto para el uso de conversiones en las respuestas al comparar.

DEBILIDADES

Está hecho en consola, pero tiene creatividad.

No se puede cancelar al momento de seleccionar 'GENERAR UN EXAMEN'.

Bitácora de trabajo

Isaac Tejeda	Edgar Jehonadab Armas De Paz
• Investigación para conocer herramientas a utilizar(1-2 días). • Investigación de funciones y lógica para el funcionamiento del código con los archivos de texto y empleo de librerías para el libre manejo y cuidado en errores(3-5 días). • Investigación del manejo de listas: tipo de lista doble enlazada, esta vista en el curso de estructuras de datos, enfocado en especial en esta y sus algoritmos de funcionamiento(1-3 días). • Manejo de archivos binarios y txt, estos vistos en el curso de programación 1, enfocado más en los archivos (.txt), éstos para respetar el formato que se pide para los exámenes(2-4 días). • Construcción del sistema de estructuras para el manejo de datos del contenido de los archivos(3-4 horas). • Empleo de funciones auxiliares, esto fue más tardado ya que fue parte del desarrollo del código y estas funciones se fueron adaptando dependiendo de las limitaciones que se tenían y pensando en que más integrar(tiempo total). • Desarrollo de las funciones para guardar las preguntas dentro de los archivos (.txt), para guardar las preguntas con el formato deseado en la carpeta de 'Exámenes'(1-5 horas). • Creación del repositorio para el control de versiones(10-15 min). • Desarrollo de la primera versión del menú en main y primera lógica de funcionamiento con botones(45 - 70 min). • Investigación y desarrollo de la función para generar exámenes, con un sistema de rellenado de información para el nombre del (.txt) que será el examen, cada reactivo y pedido de la sugerencia al final al usuario de seguir añadiendo o no, manejos en bucles y condiciones y direcciones de memoria en explorador de archivos(7 < horas). • AMBOS: CORRECCIÓN DE ERRORES Y VERIFICACIÓN DE UNA BUENA COMPILACIÓN(1 día).	• Diseño de la estructura del proyecto. Se definió que los exámenes se almacenaron en archivos .txt en una carpeta llamada Exámenes (1 día). • Investigación para hacer los menús más visualmente amigables. Se estudió cómo capturar flechas con getch() y los códigos de tecla extendidos. Para resaltar la opción seleccionada se usó código ANSI (1 día). • Diseño y funciones de los mues, la función que dibuja el menú recibe un vector de strings y el índice seleccionado, limpia la pantalla y pinta la opción seleccionada resaltandola. La función para obtener las teclas usa getch() para la lectura de las letras (1 día). • Funciones para la modificación de un examen. Emplea una función que dibuja el menú de modificación, manteniendo visibles las pregunta, las opciones y la respuesta correcta con los puntos que da y muestra un menú de acciones (2 horas). • Funciones para la aplicación de un examen. Se carga el archivo en una lista doblemente enlazada de preguntas y tras cargar el archivo muestra la pregunta actual con sus respuestas, destacando con un asterisco la elegida. El usuario puede navegar entre respuestas usando las flechas de arriba y abajo y navegar entre preguntas usando las flechas de derecha a izquierda (2 horas). • Funciones para modificar las preguntas. La función que edita las preguntas toma un puntero al nodo actual y muestra los valores existentes. Para cada campo se ofrece la posibilidad de escribir un nuevo valor o conservar el mismo valor (3 horas). • Funciones para la lista doblemente enlazada. Se implementó la función crearNodo, reservar memoria dinámica para un nodo con una pregunta y poner punteros a siguiente y atrás. La función para añadir nodo (anadirNodo), inserta el nuevo nodo al final de la lista y mantiene los enlaces bidireccionales. Ambas funciones son usadas únicamente en la carga de exámenes (1 hora).

Conclusiones

Como conclusión, podemos decir que los conocimientos adquiridos a lo largo de este curso, se fortalecieron y se quedaron más plantados en nuestro conocimiento aplicándolos en este proyecto.

Fue un curso muy formativo, aprendimos nuevas formas de resolver problemas a través de algoritmos y aplicaciones de estructuras de datos. Aprendimos a emplear el conocimiento para tener una solución más óptima a problemas de lógica de programación, las estructuras de datos nos ayudaron a tener los datos de una manera mucho más organizada y poder trabajar con ellos de manera más eficiente, óptima y cómoda.

Las estructuras de datos que funcionan como una aplicación de algoritmos, tal cual de forma parecida a una receta, con una serie de pasos ya establecida para la solución de problemas de gran complejidad, nos sirve de mucho en la programación ya que gracias a las estructuras de datos, ya tenemos soluciones establecidas por personas que se dedicaron durante días a la solución de ese problema a través de un simple algoritmo, aplicando matemáticas y lógica abstracta.

En este proyecto por ejemplo se hizo el empleo de las estructuras, las cuales mantienen un mejor orden al ordenamiento de los datos.

El empleo de los punteros, es importante contar con la ubicación de un espacio de una memoria, esto para tener una manipulación de datos mucho más cómoda.

El uso de la memoria dinámica y está para tener almacenada información en la memoria RAM, esta es muy importante que la liberes para así no tener problemas con las fugas de memoria que pueden hacer que el sistema colapse por la necesidad de espacio en la memoria.

Las listas, que combinan estas últimas 3 funciones y que existen varias, pero la que empleamos en este caso, fue una lista doble enlazada, la cual en cada nodo cuenta con 2 punteros, siguiente y atrás, esto haciendo de una forma más llevadera el control del movimiento entre nodos, claro, dependiendo de lo que se quiera hacer en operación con la lista se tienen que desarrollar distintos algoritmos.

El empleo del guardado de información en los archivos (.txt), estos son de gran importancia al momento de que queramos guardar datos de un usuario desde el mismo programa.

Cada algoritmo tiene su grado de complejidad, dependiendo de las operaciones que le ordene a hacer a la máquina.

Bibliografía

Cppreference. (s.f.). *std::vector*. <https://en.cppreference.com/w/cpp/container/vector>

Cppreference. (s.f.). *std::basic_ifstream*.
https://en.cppreference.com/w/cpp/io/basic_ifstream

Cppreference. (s.f.). *std::basic_ofstream*.
https://en.cppreference.com/w/cpp/io/basic_ofstream

Cppreference. (s.f.). *std::basic_stringstream*.
https://en.cppreference.com/w/cpp/io/basic_stringstream

Cppreference. (s.f.). *New expression*.
<https://en.cppreference.com/w/cpp/language/new>

GeeksforGeeks. (s.f.). *Doubly linked list*.
<https://www.geeksforgeeks.org/doubly-linked-list/>

Microsoft. (s.f.). *_getch, _getwch*.
<https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/getch-getwch>